

Types of Interrupts:

External Interrupt:

- Used by CPU to interact with I/O device.
- This improves system performance by allowing the CPU to execute instructions, instead of just waiting for the I/O device, while still performing the required data transfers.

2. Internal Interrupt:

- Occurs entirely within CPU, I/O devices play no role in these interrupts.
- For example, a timer built into the CPU may generate an interrupt at a pre-determined interval.

3. Software Interrupt:

- Generated by the program itself.
- It itself is like a subroutine. Eg: HLT

When we give HLT, it calls a predefined subroutine for it, which stops the program we are using.

INTERRUPT PROCESSING

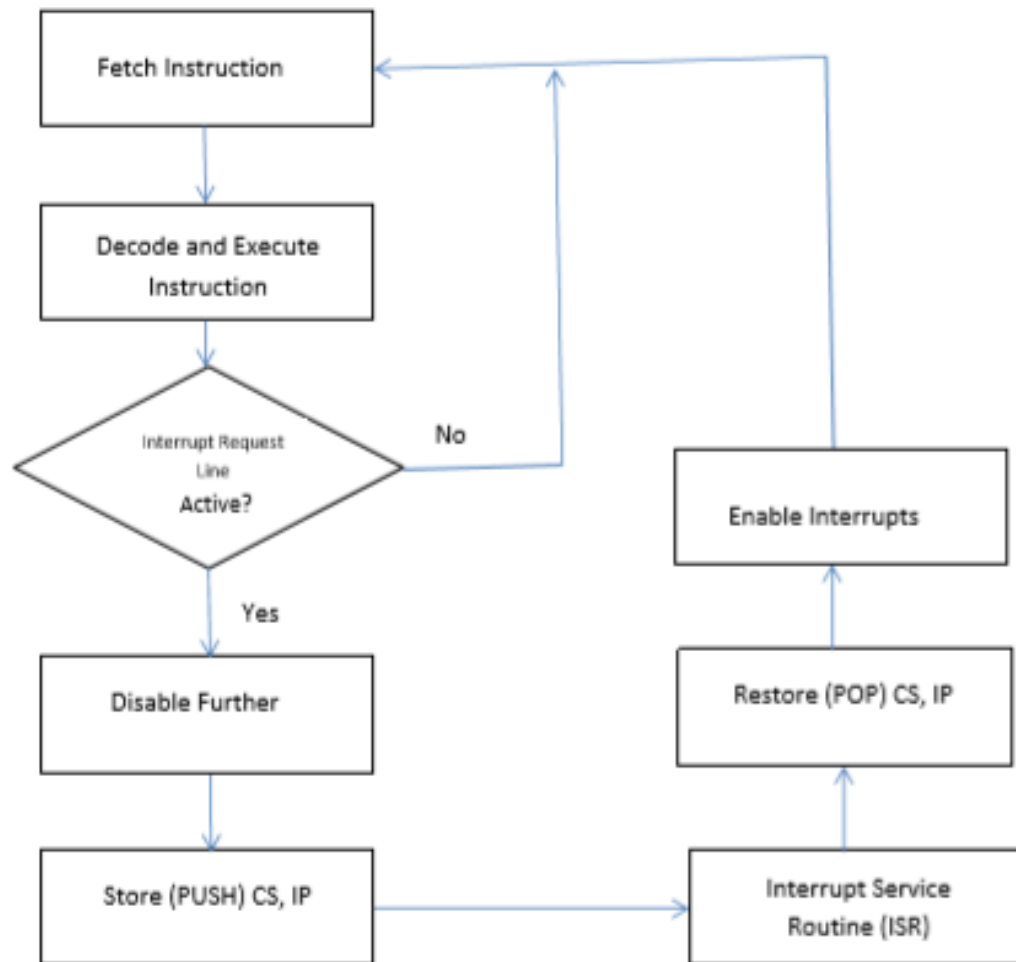
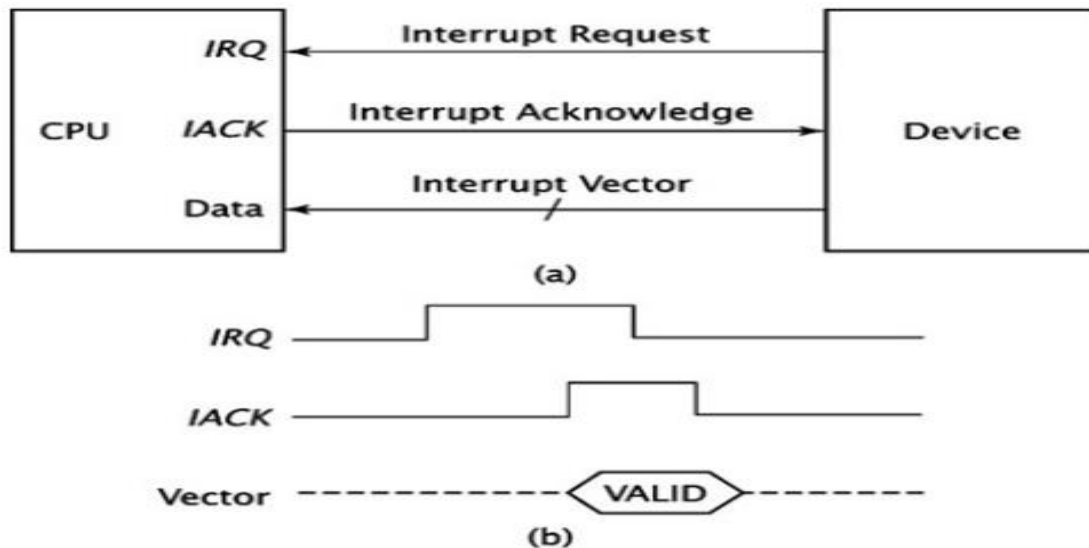


Figure: Interrupt Processing

- The processor checks for interrupts. If interrupt has occurred, processor will complete the instruction currently being executed.
- The processor will disable the further interrupts.
- The processor stores the current state of program by PUSH operation i.e. the value of flag register and CS: IP will be stored into stack by PUSH operation.
- The processor will load the address of the ISR and execute the ISR. At the end of ISR, instruction IRET is used which makes the processor return from the ISR to the original program.
- After the execution of ISR, the processor restores the previous state of program i.e. it will restore the value of flag register and CS: IP from stack by POP operation.
- The processor enables the interrupts and then starts program execution from where it has been interrupted.

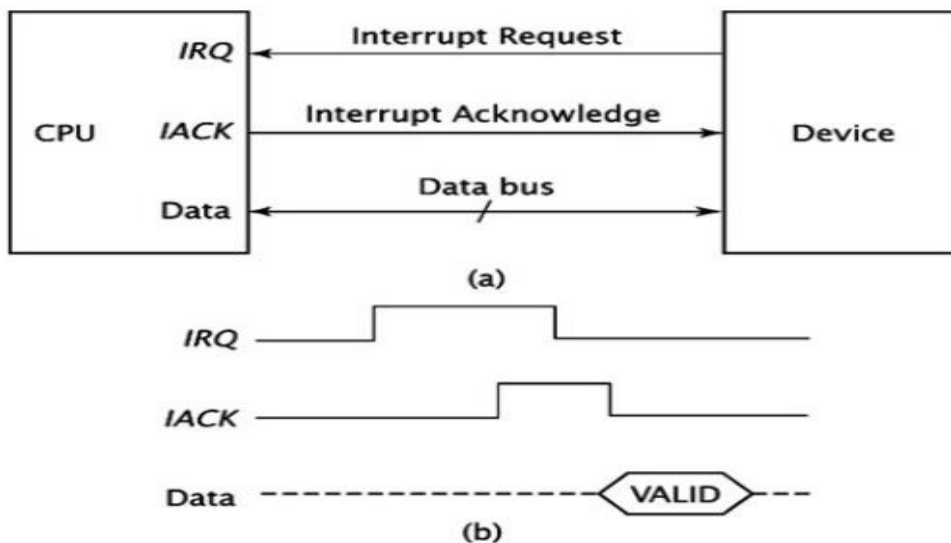
Interrupt hardware and priority

Vectored Interrupt



- CPU knows the address of ISR in advance.
- All it needs is that interrupting device sends its unique vector via a data bus and through its I/O interface to the CPU.
- CPU takes this vector, checks an interrupt table in memory and then carries correct ISR for the device.

Non-vectored interrupt

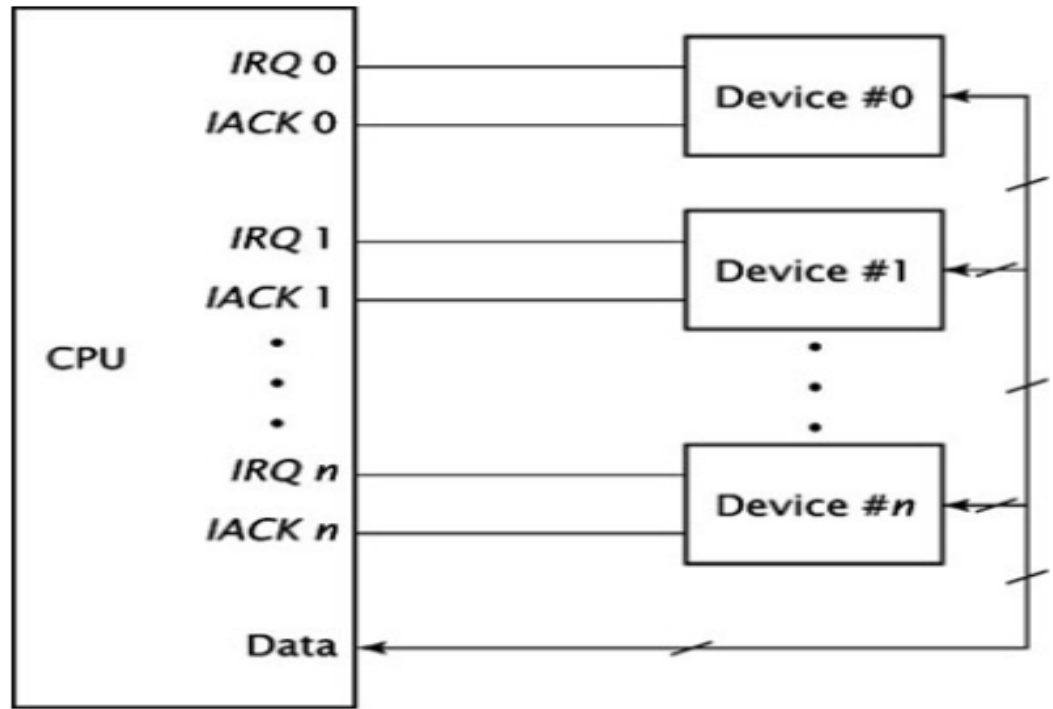


- Interrupting device never sends an interrupt vector.
- CPU receives the interrupt, and it jumps the program counter to a fixed address in hardware.

- CPU crucially does not know which device caused the interrupt without polling each I/O interface in a loop and checking status register of each I/O interface.

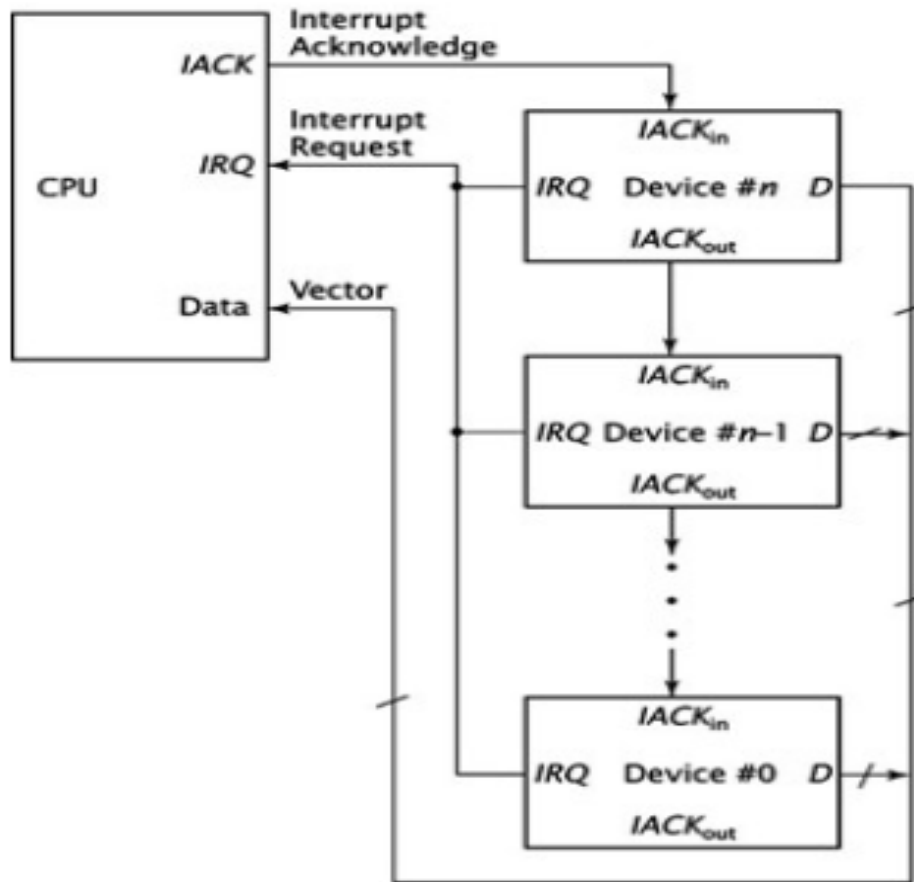
Prioritizing multiple interrupts:

Extension of non-vectored Interrupt



- IRQ_n has the highest priority and IRQ_0 has the lowest.
- If more than one device requests an interrupt, the CPU acknowledges and serves the interrupt with the highest priority first.
- This method works well when there are only a few IRQ/ACK pairs.
- As the number of interrupt increases, the number of pins needed by the CPU to accommodate these signals become prohibitive.

Daisy Chaining



- The interrupt request signals from the devices are ORed together.
- When the CPU receives an active IRQ input, it doesn't know which device generated the interrupt request.
- It sends out an acknowledgement signal and leaves it to the devices to work among themselves.
- Device #n receives the IACK signal directly from the CPU.
- If IACK is asserted, usually 1, this device has requested an interrupt; it sets its IACK_{out} signal to 0 and places its vector on the data bus.
- If this device did not request the interrupt, it sets its IACK_{out} signal to 1, thus passing it to device n-1 and so on until the IACK is asserted
- Straightforward and easy to implemented.
- The configuration is sequential, and thus can introduce hardware delays, if the chain is too long.

Implementing priority interrupts in parallel

- Implemented using priority encoder.
- Output of this encoder is the value of the highest priority device requesting an interrupt.
- Priority of the device doesn't determine the time needed to acknowledge the interrupt.

